

L9c: Logistic Regression and Numerical Optimization

The logistic model, gradient descent, and simulated annealing

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can a linear classifier report how confident it is, and how do we estimate its parameters when no formula gives them directly?

Today's concepts

- Derive the logistic regression model for binary classification: derive the logistic function from the Boltzmann distribution and use its probabilities to assign class labels.
- Estimate logistic regression parameters with gradient descent: minimize the cross-entropy loss with its gradient, and add a penalty on the size of the parameters.
- Solve constrained problems with simulated annealing: fold the constraints into the objective with penalty and barrier terms, and choose the temperature schedule.

Lecture: Logistic Regression and Numerical Optimization

Lecture and Example

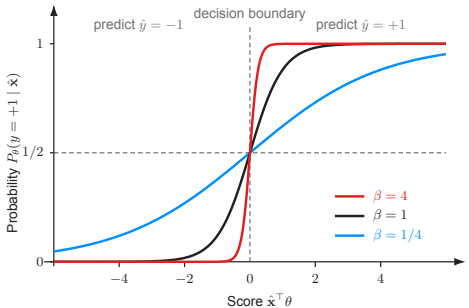
Lecture: Logistic Regression and Numerical Optimization
In this lecture, we derive logistic regression, estimate its parameters with gradient descent, and solve constrained problems with simulated annealing.

Example: Fit a Logistic Regression Classifier
In this example, we fit a logistic regression model to two overlapping clusters with gradient descent and with simulated annealing, which never computes a gradient, and compare the two answers.

Unlike the Perceptron, logistic regression predicts a class label and the probability behind it.

From the Sign Function to a Probability

The Perceptron labels an example by the sign of the score $\hat{\mathbf{x}}^\top \theta$. Logistic regression replaces the sign with a probability that rises smoothly with the score.



Each curve is the probability of $y = +1$ for one inverse temperature β. Every curve crosses $1/2$ at the decision boundary; a larger β makes the rise sharper.

Labels as States in a Boltzmann Distribution

We treat the labels $y \in \{-1, 1\}$ as states of a Boltzmann distribution conditioned on $\hat{\mathbf{x}}$, where lower-energy states are more probable. With energy $E(y, \hat{\mathbf{x}})$ and inverse temperature $\beta = 1/T > 0$:

$$P(y \mid \hat{\mathbf{x}}) = \frac{\exp(-\beta E(y, \hat{\mathbf{x}}))}{\underbrace{\sum_{y' \in \{-1,1\}} \exp(-\beta E(y', \hat{\mathbf{x}}))}_{Z(\hat{\mathbf{x}})}}$$

The partition function $Z(\hat{\mathbf{x}})$ normalizes the probabilities. For the energy, we use a linear model:

$$E(y, \hat{\mathbf{x}}) = -y(\hat{\mathbf{x}}^\top \theta).$$

A large positive score $\hat{\mathbf{x}}^\top \theta$ gives $y = +1$ the lower energy, so it is more probable; a very negative score favors $y = -1$.

Deriving the Logistic Function

Substitute the energy and let $z = \beta\,\hat{\mathbf{x}}^\top\theta$:

$$P_\theta(y \mid \hat{\mathbf{x}}) = \frac{\exp(yz)}{\exp(z) + \exp(-z)} = \frac{1}{\exp\big((1-y)z\big) + \exp\big(-(1+y)z\big)}.$$

Evaluating the two cases:

$$P_\theta(y = +1 \mid \hat{\mathbf{x}}) = \frac{1}{1 + \exp(-2\beta\,\hat{\mathbf{x}}^\top\theta)}, \qquad P_\theta(y = -1 \mid \hat{\mathbf{x}}) = \frac{1}{1 + \exp(2\beta\,\hat{\mathbf{x}}^\top\theta)}.$$

Together, these give the logistic function $\sigma(z) = 1/(1 + e^{-z})$:

$$\boxed{P_\theta(y \mid \hat{\mathbf{x}}) = \frac{1}{1 + \exp(-2\beta y\,\hat{\mathbf{x}}^\top\theta)} = \sigma(2\beta y\,\hat{\mathbf{x}}^\top\theta)}$$

The logistic function maps any real input into $(0, 1)$, and $\sigma(-z) = 1 - \sigma(z)$, so the two label probabilities sum to one.

The Inverse Temperature β

The inverse temperature $\beta > 0$ sets how confident the model is in its decisions:

- Large β (cold system): small energy differences give large probability differences. As $\beta \to \infty$, the logistic function becomes the Perceptron's hard threshold.
- Small β (hot system): the states become nearly equally probable. As $\beta \to 0$, each label's probability approaches $1/2$.

In practice, β appears only in the product $\beta\,\theta$, so we fix β and learn θ. Setting $\beta = 1/2$ gives the common form $P_\theta(y \mid \hat{\mathbf{x}}) = \sigma(y\,\hat{\mathbf{x}}^\top \theta)$.

From Probabilities to Class Labels

To classify an example, we predict the more probable label:

$$\hat{y} = \begin{cases} +1 & \text{if } P_\theta(y = +1 \mid \hat{\mathbf{x}}) \geq 1/2, \\ -1 & \text{otherwise.} \end{cases}$$

Because $\sigma(z) \geq 1/2$ exactly when $z \geq 0$, this is the Perceptron's rule $\hat{y} = \texttt{sign}(\hat{\mathbf{x}}^\top \theta)$, with the same decision boundary.

Choosing a threshold. We can instead predict $\hat{y} = +1$ when the probability is at least some threshold $t \in (0, 1)$. Lowering t labels more examples positive, so the recall and the number of false positives can only rise or stay the same.

When a missed positive is costly, as in disease screening, a threshold below $1/2$ may be better.

The Cross-Entropy Loss

For a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid i = 1, \dots, n\}$ of independent examples, we choose the parameters that make the observed labels most likely:

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} P_\theta(y_i \mid \hat{\mathbf{x}}_i) \quad \Longrightarrow \quad \log \mathcal{L}(\theta) = -\sum_{i=1}^{n} \log\Big(1 + \exp(-2\beta y_i \, \hat{\mathbf{x}}_i^\top \theta)\Big).$$

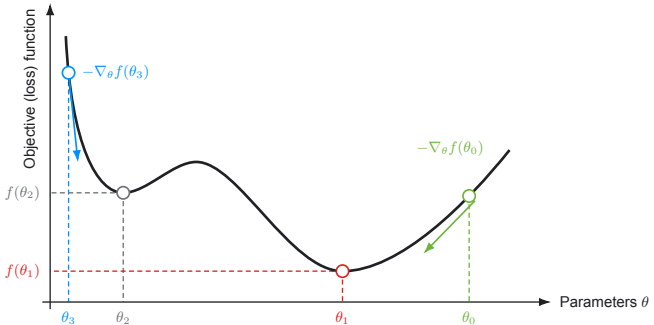
Taking the log turns the product into a sum. Minimizing the negative log-likelihood, the cross-entropy loss, gives the estimate:

$$\boxed{J(\theta) = \sum_{i=1}^{n} \log\Big(1 + \exp(-2\beta y_i \, \hat{\mathbf{x}}_i^\top \theta)\Big)}, \qquad \hat{\theta} = \arg\min_\theta J(\theta).$$

Unlike least squares, $\nabla J(\theta) = \mathbf{0}$ has no closed-form solution, so we minimize $J(\theta)$ numerically.

Gradient Descent and Local Minima

The gradient points in the direction of steepest increase, so gradient descent steps opposite the gradient.



Started at θ_0, it reaches the lowest point θ_1; started at θ_3, it stops at θ_2, a local minimum.

Gradient of the Cross-Entropy Loss

Let $u_i = 2\beta y_i \, \hat{\mathbf{x}}_i^\top \theta$, so term i of the loss is $\log(1 + e^{-u_i})$. By the chain rule:

$$\nabla_\theta \log(1 + e^{-u_i}) = \left(\frac{-e^{-u_i}}{1 + e^{-u_i}} \right) \nabla_\theta u_i = -(1 - \sigma(u_i))\, 2\beta y_i \, \hat{\mathbf{x}}_i.$$

Summing over the training examples:

$$\boxed{\nabla J(\theta) = -2\beta \sum_{i=1}^{n} (1 - \sigma(u_i))\, y_i \, \hat{\mathbf{x}}_i}$$

The factor $1 - \sigma(u_i)$ is the probability the model gives to the wrong label for example i, so each step adds every $y_i \, \hat{\mathbf{x}}_i$, weighted by this probability.

The Perceptron update uses only the mistakes, each with full weight; here, confidently correct examples contribute almost nothing.

Gradient Descent Algorithm

Initialize: Choose $\theta^{(0)}$ (random or zeros), a learning rate $\alpha(k) > 0$, a tolerance $\epsilon > 0$, and `maxiter`. Set `converged` $\leftarrow$ `false` and $k \leftarrow 0$.

While not `converged` **do**:

1. Compute the direction $\mathbf{d} = -\nabla J(\theta^{(k)})$.
2. Update $\theta^{(k+1)} = \theta^{(k)} + \alpha(k)\,\mathbf{d}$.
3. Check the stopping criterion:
   - If $\|\theta^{(k+1)} - \theta^{(k)}\|_2 \leq \epsilon$, **then** set `converged` $\leftarrow$ `true` and return $\theta^{(k+1)}$.
   - If $k + 1 \geq$ `maxiter`, **then** return $\theta^{(k+1)}$ and warn that it did not converge.
4. Increment $k \leftarrow k + 1$, and update $\alpha(k)$, if applicable.

Too large a learning rate may overshoot the minimum; too small a rate converges slowly. We can also stop when $\|\nabla J(\theta^{(k)})\|_2 \leq \epsilon$.

Convexity of the Loss

A function f is convex if, for any inputs $\mathbf{a}, \mathbf{b}$ and any weight $t \in [0, 1]$:

$$f(t\,\mathbf{a} + (1-t)\,\mathbf{b}) \leq t\,f(\mathbf{a}) + (1-t)\,f(\mathbf{b}).$$

The line segment connecting any two points on the graph lies on or above the graph. For example, x^2 and e^x are convex, while $-x^2$ is not.

A convex function has no local minimum like θ_2 in the figure: any local minimum is a global minimum.

Each term of the cross-entropy loss is a convex function of u_i, which is linear in θ, so $J(\theta)$ is convex.

How Good Is Gradient Descent?

- A safe learning rate: the gradient is Lipschitz, so with $\alpha \leq 1/L$, every step lowers the loss unless the gradient is zero:

$$\|\nabla J(\mathbf{a}) - \nabla J(\mathbf{b})\|_2 \leq L\,\|\mathbf{a} - \mathbf{b}\|_2, \qquad L = \beta^2 \|\hat{\mathbf{X}}\|_2^2,$$

  where $\|\hat{\mathbf{X}}\|_2$ is the largest singular value of the data matrix.

- Slowing down over time: when the loss has a minimizer, the gap to the optimal loss after k steps shrinks like $1/k$. We sprint at first, then settle into a steady jog.

- When there is no minimum: on linearly separable data, scaling up a separating θ lowers every term of the loss, so θ keeps growing. Regularization fixes this.

Advanced: Momentum methods, Newton's method, and constrained gradient descent

Regularization: Preventing Overfitting

To keep the model from following the training data too closely, we add a ridge penalty on the size of the parameters:

$$\boxed{J_\delta(\theta) = J(\theta) + \frac{\delta}{2}\,\|\theta\|_2^2}, \qquad \nabla J_\delta(\theta) = \nabla J(\theta) + \delta\,\theta,$$

where $\delta \geq 0$ is the regularization parameter. We minimize J_δ with gradient descent.

With $\delta > 0$, the loss is strongly convex: it has exactly one minimizer, even for separable data, and each step with $\alpha \leq 1/(L + \delta)$ shrinks the gap to the minimum loss by at least a constant fraction.

- Scale the features to comparable magnitudes, so the penalty treats the parameters fairly.
- Choose δ by cross-validation, as we did for ridge regression.

Constrained Problems

Many engineering problems restrict their decision variables, for example, to fractions that must be nonnegative and sum to one. We minimize a nonlinear objective $f(x)$ subject to m inequality and p equality constraints:

$$\min_{x \in \mathbb{R}^n} \; f(x) \quad \text{s.t.} \quad \begin{cases} g_i(x) \leq 0, & i = 1, \dots, m, \\ h_j(x) = 0, & j = 1, \dots, p. \end{cases}$$

Logistic regression has no constraints on θ. Simulated annealing, like gradient descent, works on an unconstrained objective, so we fold the constraints into it.

Advanced: The Karush--Kuhn--Tucker conditions and a gradient method for this problem

Penalty and Barrier Terms

A penalty grows with the size of a constraint violation. A barrier grows without bound as x approaches the edge of the region where the inequality constraints hold:

$$P_{\mu,\rho}(x) = f(x) \; \underbrace{- \mu \sum_{i=1}^{m} \ln(-g_i(x))}_{\text{barrier term}} \; + \; \underbrace{\frac{1}{2\rho} \sum_{j=1}^{p} \left[h_j(x)\right]^2}_{\text{penalty term}}, \qquad \mu > 0, \; \rho > 0.$$

The logarithmic barrier keeps the solution strictly inside the region. We decrease both parameters over the iterations:

- Barrier weight: as $\mu \to 0$, the barrier fades inside the region, so the solution can approach its edge, where the optimum often lies.
- Penalty weight: as $\rho \to 0$, the coefficient $1/\rho$ grows, enforcing the equality constraints more strictly.

Simulated Annealing

Simulated annealing (Kirkpatrick, Gelatt, and Vecchi, 1983) is a derivative-free method inspired by annealing in metallurgy: heating a metal and cooling it slowly so its atoms settle into a low-energy arrangement.

It accepts both improving and non-improving moves, controlled by a temperature T. A move that raises the objective by $\Delta P > 0$ is accepted with probability:

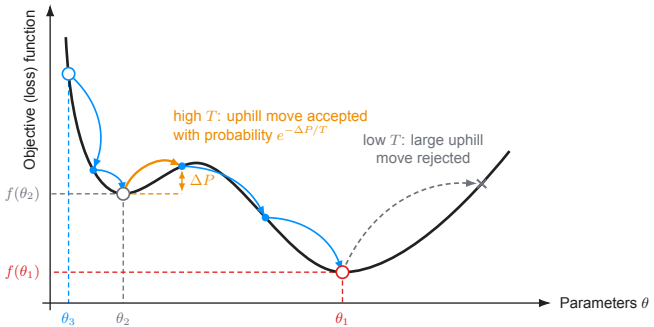
$$p_{\text{accept}} = \exp(-\Delta P / T),$$

the Boltzmann factor from our derivation with $\beta = 1/T$.

- At a high temperature, uphill moves are often accepted.
- As T decreases, they become rare.

Escaping a Local Minimum

Started at θ_3, simulated annealing moves downhill to θ_2, where gradient descent stops.



At a high temperature, it accepts an uphill move, then moves downhill to θ_1; at a low temperature, it rejects a large uphill move.

Simulated Annealing: Setup

Initialize: Given:

- an initial solution x_0 with $g_i(x_0) < 0$ for all i,
- penalty parameters $\mu, \rho > 0$ with update factors $\tau_\mu, \tau_\rho \in (0, 1)$,
- an initial temperature $T \leftarrow T_\circ$ and a minimum temperature $T_{\min}$, with $T_\circ > T_{\min} > 0$,
- a cooling rate $\alpha \in (0, 1)$, a step size $\eta > 0$, and the number of iterations K per temperature.

Set `converged` $\leftarrow$ `false`, the best solution $x^\star \leftarrow x_0$, and the current solution $x_c \leftarrow x_0$.

The equality constraints enter only through the penalty term, so at the end we check how close each $h_j(x^\star)$ is to zero.

Simulated Annealing: The Loop

While not `converged` **do**:

1. For $k = 1$ to K **do**:
   - Generate a candidate $x' \leftarrow x_c + \eta \cdot \texttt{randn}(\texttt{size}(x_c))$.
   - If $g_i(x') \geq 0$ for any i, **then** reject it; the barrier is undefined there.
   - Compute $\Delta P \leftarrow P_{\mu,\rho}(x') - P_{\mu,\rho}(x_c)$. If $\Delta P < 0$, accept: $x_c \leftarrow x'$. Otherwise, draw $u \sim \texttt{Uniform}(0, 1)$ and accept if $u \leq \exp(-\Delta P / T)$.
   - If $P_{\mu,\rho}(x_c) < P_{\mu,\rho}(x^\star)$, **then** set $x^\star \leftarrow x_c$, with both values at the current μ, ρ.
2. Update the penalty parameters: $\mu \leftarrow \tau_\mu \, \mu$ and $\rho \leftarrow \tau_\rho \, \rho$.
3. If $T \leq T_{\min}$, **then** set `converged` $\leftarrow$ `true` and return $x^\star$. Otherwise, cool: $T \leftarrow \alpha \, T$.

Here, `randn` draws independent normal random numbers with mean zero and variance one.

Selecting $T_\circ$ and K

Simulated annealing is a heuristic: it does not guarantee the global minimum, and its result depends on the temperature schedule.

Sample-and-set rule for $T_\circ$. From x_0, generate $M = 100$ candidates, discard any that violate an inequality constraint, and average the uphill changes $\Delta P_j > 0$ to get $\overline{\Delta P}_+$. Choose the probability $p_\circ \in (0.6, 0.9)$ of accepting an average uphill move at the start, and set:

$$T_\circ = -\frac{\overline{\Delta P}_+}{\ln p_\circ}, \qquad \text{so that} \qquad \exp(-\overline{\Delta P}_+ / T_\circ) = p_\circ.$$

Heuristic for K. Start with $K = \lceil c\, n \rceil$, where n is the number of decision variables and $c \in [10, 50]$. After each temperature, compute the acceptance rate $\hat{p}$: if $\hat{p} > 0.8$, set $K \leftarrow \lceil 0.75K \rceil$; if $\hat{p} < 0.2$, set $K \leftarrow \lceil 1.5K \rceil$.

Example: Fit a Logistic Regression Classifier

In this example, we fit one logistic regression model to two overlapping clusters of points with both methods.

1. Fit the model with gradient descent, with the learning rate $\alpha = 1/L$ computed from the training data, and watch the loss fall.
2. Draw the probability contours and the decision boundary, and classify the testing points at several thresholds.
3. Fit the same model with simulated annealing, choosing $T_\circ$ with the sample-and-set rule, and compare the two answers.

Simulated annealing reaches nearly the same parameters without the gradient, at the cost of many more loss evaluations.

Example: Fit a logistic regression classifier with gradient descent and simulated annealing

Lab: Logistic Regression for Banknotes

In the lab, we train a logistic regression classifier with gradient descent on the banknotes from the Perceptron example:

- complete the training loop for the regularized cross-entropy loss,
- evaluate the classifier with a confusion matrix and compare it with the classical and pocket Perceptron on the same banknotes, and
- use the predicted probabilities to decide which banknotes a person should check by hand.

The probability tells us which banknotes the classifier is unsure about.

Lab: Logistic regression with gradient descent on the banknotes

Summary

We derived logistic regression from the Boltzmann distribution, fit it with gradient descent, and solved constrained problems with simulated annealing.

Key takeaways

- **The logistic regression model.** We treated the labels as Boltzmann states and derived the logistic function; predicting the more probable label recovers the Perceptron's decision boundary.
- **Parameter estimation with gradient descent.** We minimized the cross-entropy loss, weighting each example by its wrong-label probability; a ridge penalty gives a unique minimizer.
- **Constrained problems and simulated annealing.** We folded constraints in with penalty and barrier terms; accepting some uphill moves lets the search escape local minima.

The example and lab apply these ideas to synthetic clusters and to the banknotes.